FinaMente

1. Identificación del problema del track

Track seleccionado

Grupo Salinas / Banco Azteca — Innovación tecnológica para la educación e inclusión financiera en México
Convocatoria: Genius Arena Hackathon 2026

Problema principal

La falta de conocimiento sobre el funcionamiento de las tarjetas de crédito provoca que jóvenes usuarios cometan errores financieros que derivan en endeudamiento temprano.

Contexto del track

El acceso a tarjetas de crédito en México se ha vuelto cada vez más sencillo, permitiendo que jóvenes las obtengan desde etapas tempranas sin contar con conocimientos suficientes sobre su funcionamiento. Esta situación se ve agravada por la falta de educación financiera formal, lo que provoca que muchos usuarios desconozcan conceptos clave como intereses, pagos mínimos y financiamiento, aumentando la probabilidad de atrasos y sobreendeudamiento desde los primeros usos del crédito.

Evidencias

- 3 de cada 10 personas con crédito en México han tenido atrasos en sus pagos, y el 28.6% de esos casos corresponde a personas entre 18 y 29 años. [1]
- 6.6 millones de jóvenes presentan niveles altos de estrés financiero y 9.5 millones niveles moderados. [2]
- La falta de educación financiera limita la comprensión de productos financieros y dificulta la toma de decisiones informadas a largo plazo. [3]

Alcance

La solución se limita a la simulación educativa del comportamiento financiero asociado al uso de tarjetas, cubriendo únicamente los conceptos fundamentales: fechas de corte y pago límite, tasa de interés, CAT, pago mínimo, pago total, MSI, compras a plazos, disposición de efectivo y uso básico de débito.

2. Usuario objetivo y escenario de uso

Usuario objetivo: estudiantes y jóvenes que están por obtener su primera tarjeta de crédito o que muestran interés en adquirirla por primera vez. Este perfil fue seleccionado porque representa la etapa en la que se forman los primeros hábitos financieros y donde los errores iniciales pueden tener consecuencias a largo plazo en el historial crediticio.

Escenario de uso:

1. Un joven muestra interés en obtener una tarjeta de crédito en Banco Azteca u otra institución financiera.
2. Antes de adquirir el producto, se le invita a usar FinaMente.
3. Selecciona un perfil que representa su situación actual: estudiante dependiente, estudiante que trabaja, joven con empleo estable o ingresos esporádicos.
4. El sistema ejecuta una simulación interactiva de 6 meses donde el usuario toma decisiones de pago reales.
5. Al finalizar, recibe retroalimentación personalizada basada en sus decisiones y una proyección de su situación financiera.

Criterios de éxito: el usuario comprende conceptos clave como intereses, pagos mínimos y fechas de pago; puede visualizar el impacto de sus decisiones; y recibe feedback diferenciado: si tuvo buen desempeño se le invita a dar el paso real, si no, el sistema le explica los errores cometidos y sus riesgos.

---

3. Propuesta de solución

FinaMente es una aplicación web gamificada de simulación financiera dirigida a estudiantes y jóvenes que están por obtener su primera tarjeta de crédito. Funciona como un juego de rol por etapas donde el usuario enfrenta gastos simulados durante 6 meses y decide cómo pagarlos: tarjeta de crédito, débito, MSI o contado. Corre desde cualquier navegador, utiliza datos simulados y no requiere conexión con sistemas bancarios reales.

¿Qué hace? Permite al usuario seleccionar un perfil con características financieras predeterminadas y enfrentar gastos mensuales simulados durante 6 etapas. En cada gasto decide cómo pagarlo; sus decisiones afectan variables como deuda, intereses, crédito disponible e historial crediticio, generando consecuencias visibles al cierre de cada etapa. Al finalizar, el sistema entrega retroalimentación personalizada generada por IA.

¿Qué resultado produce? Los errores financieros ocurren dentro de una simulación segura, facilitando el aprendizaje mediante experiencia directa antes de que tengan consecuencias reales. Esto reduce la probabilidad de mal uso del crédito y mejora la preparación del usuario antes de obtener una tarjeta.

Diferenciador: a diferencia de videos, cursos o calculadoras financieras, FinaMente gamifica específicamente la decisión de *cómo pagar*, que es el momento crítico donde ocurren los errores más comunes. Las soluciones existentes (Credit Karma, Bank of America Better Money Habits, Kredit Quest) gamifican trivia, ahorro o puntaje crediticio general — no la decisión de pago concreta. Cada mecánica del juego existe para ilustrar un concepto financiero, no como decoración.

---

4. Justificación Tecnológica

FinaMente combina un **frontend React** con un **backend Spring Boot** cuya función es exclusivamente recolectar datos anónimos de partidas y generar retroalimentación personalizada mediante IA. Toda la lógica del juego corre en el navegador; el backend no interviene durante la partida, solo al finalizarla.

Stack tecnológico

Componente	Tecnología	Justificación resumida
Interfaz y estado	**React**	Gestión reactiva de múltiples pantallas y datos en tiempo real mediante componentes reutilizables.
Gráficos y animaciones	**Three.js**	Animaciones de impacto financiero directamente en el navegador, sin plugins.
Lógica del juego	**JavaScript (módulos)**	Motor de reglas determinista: CAT, intereses, game over, historial. Centralizado, consistente y testeable.
Backend	**Spring Boot**	Endpoints para registrar datos anónimos y mediar la llamada a OpenAI. Claves de API fuera del cliente.
Retroalimentación IA	**API de OpenAI**	Genera feedback educativo personalizado a partir del historial de decisiones del jugador.
Base de datos	**MongoDB**	Registros anónimos de partidas para analizar patrones de error e iterar el diseño.

> 📎 *Ver tabla completa con justificación extendida, consecuencias de eliminación y beneficios por dimensión (escalabilidad, mantenibilidad, seguridad, velocidad de desarrollo) en **Anexo 3 — Stack detallado**.*

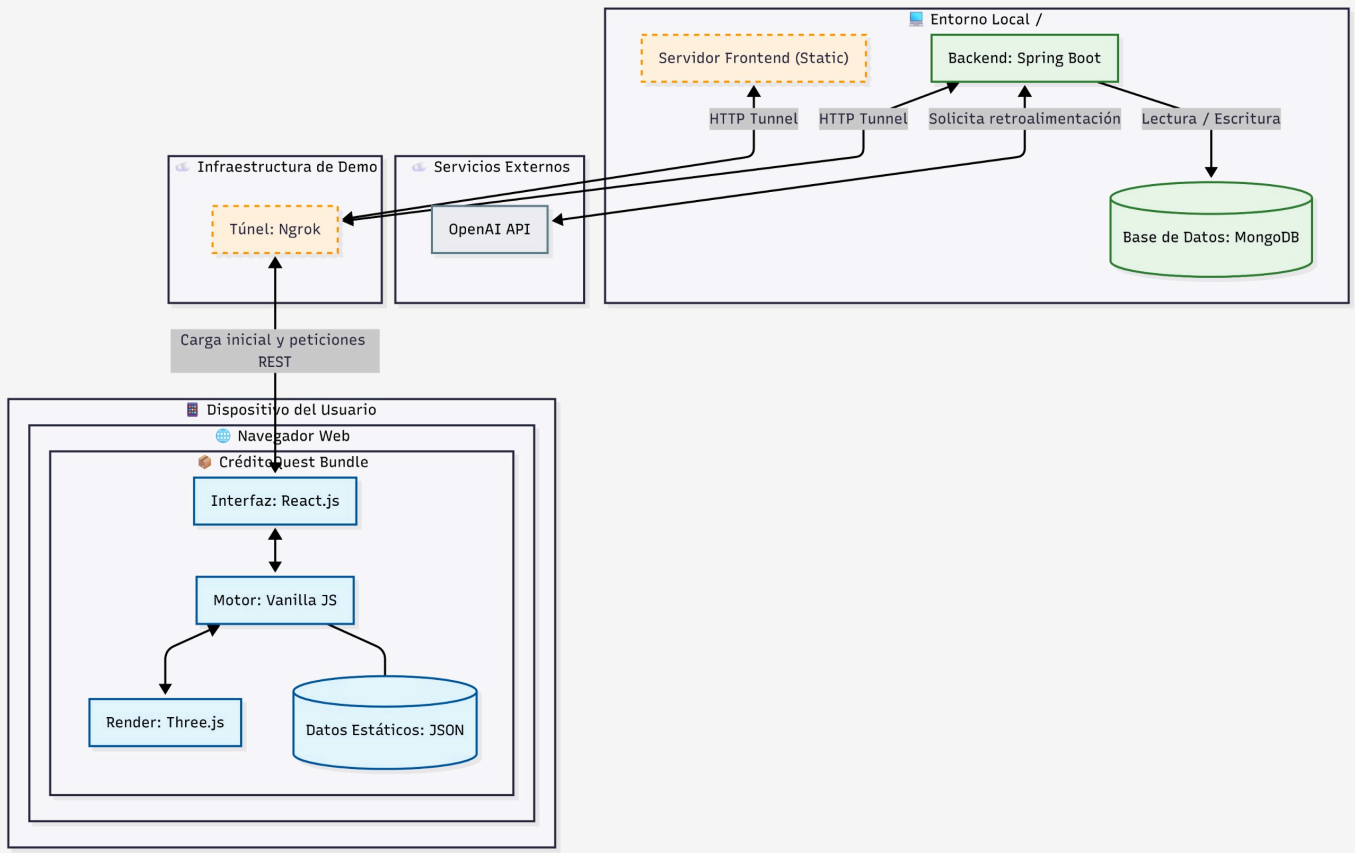
Datos

No se recolecta ningún dato personal del usuario. Los datos del juego son estáticos (perfiles basados en INEGI/ENIF, parámetros financieros de BANXICO/Condusef). Los registros en MongoDB son 100% anónimos: solo contienen decisiones de juego y resultados, sin identificadores de ningún tipo.

---

5. Arquitectura y flujo técnico

Diagrama de Despliegue



Flujo de datos: el usuario toma decisiones → React las envía al motor JS → el motor aplica reglas financieras → Three.js renderiza el impacto visual → React actualiza la UI. Al finalizar los 6 stages, el historial se envía a Spring Boot → se guarda en MongoDB → se construye un prompt y se consulta OpenAI → el feedback regresa al cliente.

Aspecto	Decisión
Latencia	100% client-side durante el juego. La única llamada de red ocurre al finalizar la partida.
Trazabilidad	Historial en memoria durante la partida; persistido de forma anónima en MongoDB al concluir.
Seguridad	Sin datos personales. Claves de OpenAI aisladas en el backend. Ngrok solo para demo.
Accesibilidad	Layout móvil-primero, interacciones táctiles, sin dependencias de red durante el juego.

> 📎 *Ver diagramas de arquitectura general y secuencia completos en **Anexo 1** y **Anexo 2**.*

---

6. Viabilidad e Impacto

Funcionalidades del MVP: selección de perfil financiero · simulación de 6 stages con motor de reglas en tiempo real · retroalimentación final generada por IA · interfaz móvil-responsiva.

Métricas de impacto (KPIs)

Métrica	Cómo se mide	Por qué importa
Tasa de finalización	Partidas iniciadas vs. completadas registradas en MongoDB	Valida usabilidad y retención.
Decisiones correctas	Análisis agregado de registros anónimos por stage	Evalúa aprendizaje práctico.
Curva de aprendizaje	Calidad de decisiones en stages 1-2 vs. 5-6	Demuestra mejora intra-sesión.
Errores críticos por perfil	Eventos de riesgo segmentados por perfil en MongoDB	Identifica factores de riesgo para iterar el juego.

Supuestos medibles: si ≥70% completa la simulación → usabilidad validada. Si las decisiones correctas mejoran de ~50% en el stage 1 a ~75% en el stage 6 → mejora educativa del 25% demostrada.

Costos estimados (MVP/demo)

Ngrok: $0 (capa gratuita) · **MongoDB Atlas:** $0 (clúster M0, 512 MB) · **OpenAI GPT-4o-mini:** ~$0.25 USD / 1,000 partidas.

Riesgos y mitigación

- **Dependencia de OpenAI:** si la API no responde, el juego sigue siendo funcional; solo se omite el feedback de IA.
- **Rendimiento visual:** si Three.js falla, la lógica JS sigue corriendo; el juego se degrada elegantemente.
- **Privacidad:** sin datos personales en ninguna capa; las claves de API nunca llegan al cliente.

---

7. Equipo

Integrante	Rol	Responsabilidades
Joni	Analista y Tester	Definición de reglas, estructuración lógica del juego, pruebas del motor y endpoints.
Omar	Desarrollador Backend, Lógica y Despliegue	Motor JS, API REST en Spring Boot, integración MongoDB y OpenAI, despliegue con Ngrok.
Diana	Desarrolladora Frontend y Storytelling	Diseño visual, UX y narrativa del pitch.
Álvaro	Desarrollador Frontend (React + Three.js)	Interfaz interactiva, visualización gráfica, conexión UI–backend.
Alexis	Desarrollador Full Stack	Refuerza cualquier módulo crítico y puede cubrir múltiples roles simultáneamente.

El equipo trabajará por módulos funcionales en paralelo con control de versiones en GitHub, con pruebas de integración al cierre de cada fase antes de avanzar.

> 📎 *Ver plan de trabajo detallado por fases en **Anexo 4 — Plan de trabajo**.*

Referencias

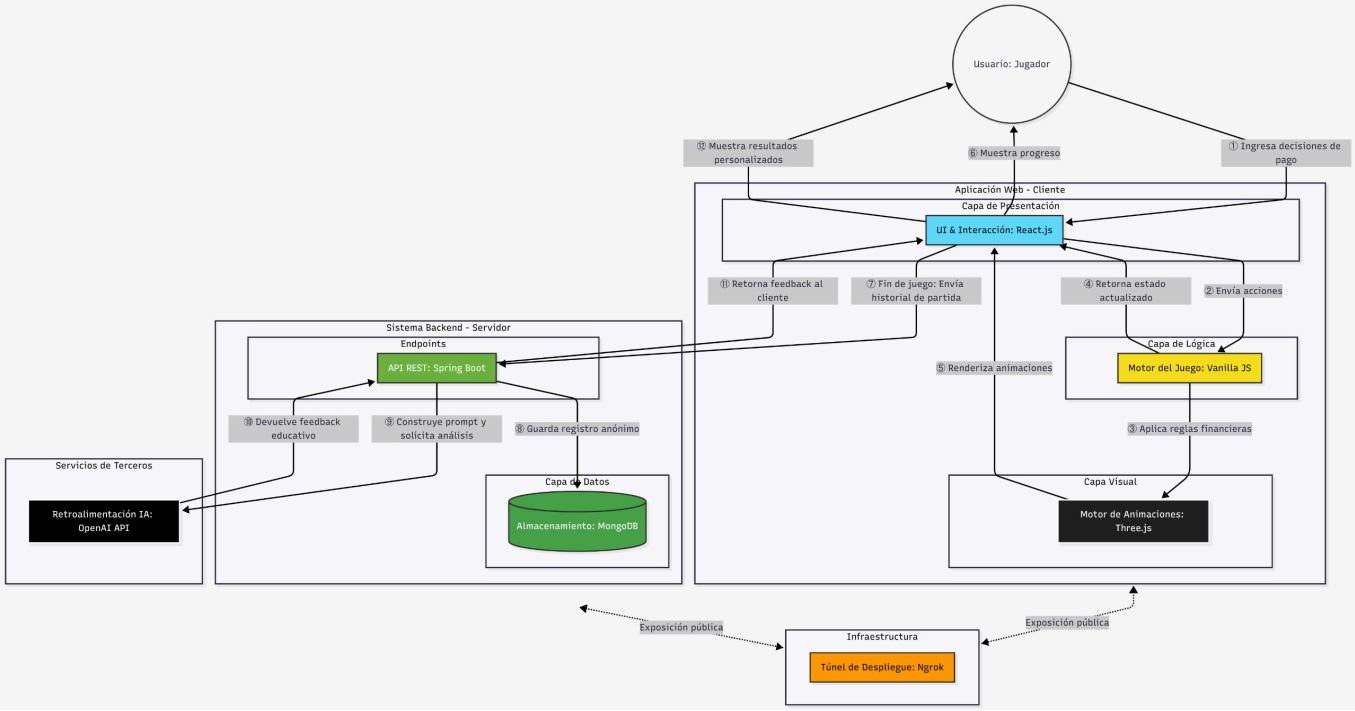
> [1] A. Angel Rodríguez, ZonaDocs, 8 abril 2025. https://www.zonadocs.mx/2025/04/08/credito-facil-deudas-dificiles-falta-de-educacion-financiera-genera-endeudamiento-de-jovenes/
> [2] F. Franco, El Economista, 7 noviembre 2025. https://www.eleconomista.com.mx/finanzaspersonales/70-jovenes-mexico-vive-estres-financiero-20251107-785428.html

[3] CONDUSEF. https://www.condusef.gob.mx/index.php/indicadores-estadisticos?idc=228&idcat=4&p=contenido

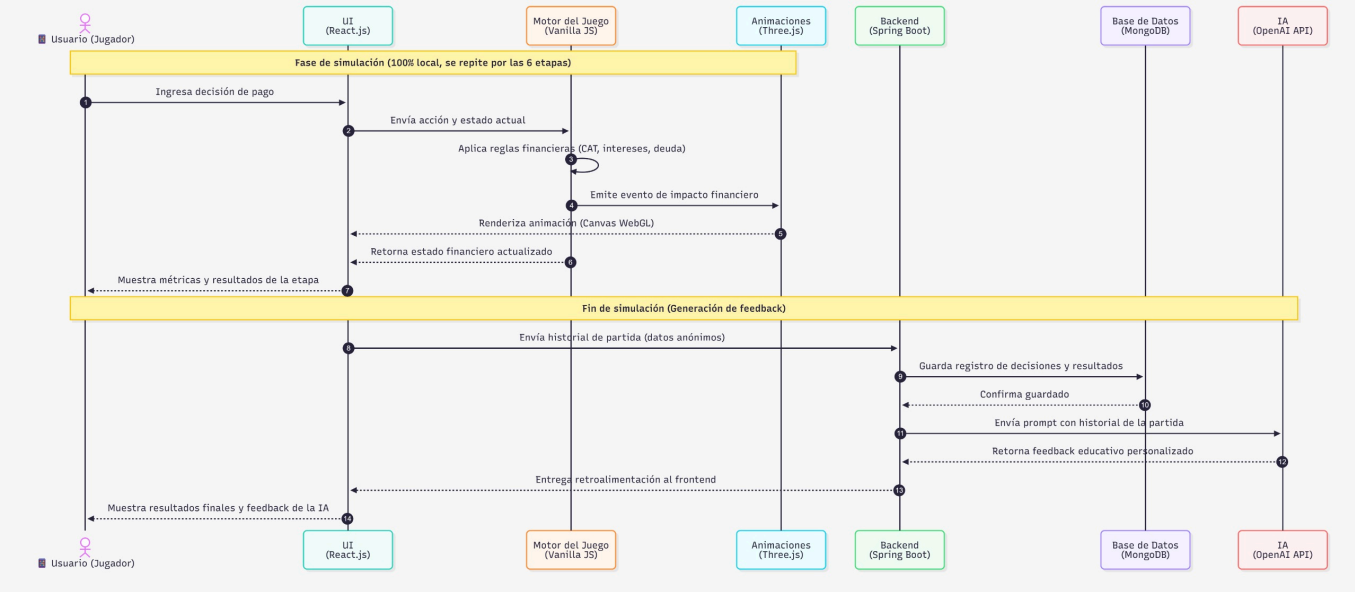
---

Anexos

Anexo 1 — Diagrama de Arquitectura General



Anexo 2 — Diagrama de Secuencia



Anexo 3 — Stack detallado

Componente	Tecnología	Por qué es necesaria	Qué pasa si se quita
Interfaz y estado	**React**	Gestión reactiva de múltiples pantallas y datos en tiempo real mediante componentes reutilizables.	La lógica de UI se dispersa en manipulación manual del DOM, propensa a errores y volviéndose difícil de mantener.
Gráficos y animaciones	**Three.js**	Renderiza efectos visuales animados directamente en el navegador, sin plugins.	El juego sigue siendo funcional pero pierde el componente visual que lo distingue de una calculadora interactiva.
Lógica del juego	**JavaScript (módulos)**	Motor de reglas determinista puro: cálculo de intereses con CAT, condición de game over, evolución del historial y desbloqueo de habilidades. Al ser lógica centralizada, produce resultados consistentes y es fácil de probar.	Sin un motor centralizado, la lógica financiera se dispersa entre componentes y genera inconsistencias entre pantallas.
Datos	**JSON / constantes JS**	Perfiles de jugador, catálogo de gastos por stage y parámetros financieros de referencia definidos como datos estáticos. No se requiere conexión externa durante el juego.	El sistema no tendría base para generar escenarios financieros.
Backend	**Spring Boot**	Expone dos endpoints: uno para recibir y almacenar datos anónimos de partida, y otro para consultar la API de OpenAI y devolver la retroalimentación final al jugador. Mantiene las claves de API fuera del cliente.	Sin backend, las claves de OpenAI quedarían expuestas en el frontend y no sería posible registrar datos de uso de forma centralizada.
Retroalimentación IA	**API de OpenAI**	Al terminar la partida, el backend construye un prompt con las decisiones tomadas por el jugador y solicita a OpenAI una retroalimentación	La pantalla de resultados quedaría reducida a mensajes genéricos predefinidos, perdiendo el valor

Componente	Tecnología	Por qué es necesaria	Qué pasa si se quita
		personalizada, clara y educativa.	diferenciador del feedback adaptado a cada partida.
Base de datos	**MongoDB**	Almacena registros anónimos de partidas: perfil seleccionado, decisiones de pago por stage y resultado final. Permite analizar patrones de error comunes entre usuarios para iterar el diseño del juego.	No habría datos agregados para medir el impacto educativo de la solución ni para respaldar decisiones de mejora.
Despliegue (demo)	**Ngrok**	Crea un túnel HTTPS desde el servidor local hacia una URL pública, permitiendo que el jurado pruebe el juego desde sus propios dispositivos sin configurar infraestructura en la nube.	La demo queda restringida a la pantalla del presentador, reduciendo el impacto del pitch.

Componente	Escalabilidad	Mantenibilidad	Seguridad	Velocidad de desarrollo
React	Nuevos stages como componentes sin tocar los existentes.	Componentes aislados; errores localizados sin afectar el resto.	Estado reactivo evita inconsistencias de datos en UI.	Componentes reutilizables aceleran el desarrollo visual.
Three.js	Efectos visuales desacoplados de la lógica del juego.	Capa visual independiente, rediseñable sin tocar reglas financieras.	Sandbox del navegador, sin implicaciones directas.	Documentación madura; efectos de impacto en pocas líneas.
JS (módulos)	Nuevos conceptos financieros = nuevos módulos.	Testeable de forma unitaria sin depender de la UI.	Ejecución 100% local en el cliente, sin exposición de red.	Sin configuración adicional de compilación.
JSON / JS	Nuevos perfiles o stages editando un archivo.	Datos separados del código; sin riesgo de romper lógica.	Sin superficie de ataque; sin consultas externas durante el juego.	Cambios en parámetros financieros visibles sin recompilar.

Componente	Escalabilidad	Mantenibilidad	Seguridad	Velocidad de desarrollo
Spring Boot	Escala con configuración estándar REST.	Frontend y backend evolucionan de forma independiente.	Claves de API nunca expuestas al navegador del cliente.	Autoconfiguración; servidor funcional con mínima configuración.
OpenAI API	Prompt enriquecible con más variables sin cambiar arquitectura.	Ajuste del feedback editando solo el prompt.	Comunicación siempre mediada por el backend.	Elimina el desarrollo de un motor de NLP propio.
MongoDB	Sin migraciones de esquema al agregar campos.	Esquema flexible adaptable sin downtime.	Sin datos personales almacenados.	JSON mapea directo a objetos del juego.
Ngrok	Solo para demo; no aplica a producción.	Sin impacto en el código.	No expone infraestructura productiva.	Un solo comando; sin configuración de DNS ni certificados.

Anexo 4 — Plan de trabajo

Fase	Módulo	Responsable(s)	Entregable
1	Definición de reglas y arquitectura	Joni + Omar	Documentación técnica y contratos de API
2	Estructura base y base de datos	Joni + Omar + Alexis	Repositorios base y esquemas de BD
3	Motor JS + Backend	Omar + Alexis	Primera demo funcional y API activa
4	Testing de lógica y endpoints	Joni	Informe de pruebas
5	Interfaz React e integración	Álvaro + Alexis	Segunda demo funcional
6	Three.js + prompt OpenAI	Diana + Álvaro + Omar	Informe de pruebas visuales y de IA
7	Integración total	Omar + Álvaro + Alexis	Tercera demo funcional (flujo completo)

Fase	Módulo	Responsable(s)	Entregable
8	Testing end-to-end	Joni	Informe de pruebas general
9	Deploy	Omar + Alexis	Aplicación desplegada (Ngrok / Cloud)
10	Pitch y presentación	Diana	Slides finales